

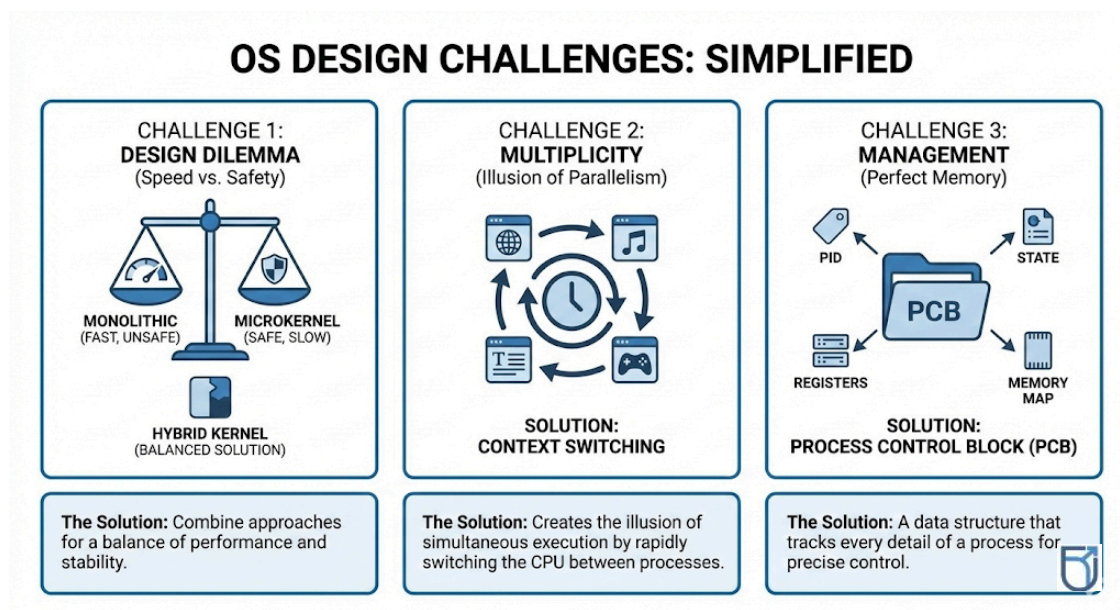
# Lecture 02 NOTES-Operating Systems

## Foundations II & Process Concept

### OS Design Challenges

#### Challenge 1: The Design Dilemma (Speed vs. Safety)

- **The Core Conflict:** The fundamental question driving OS design is how to build a system that is both **FAST** and **SAFE**. This choice shapes the entire architecture.
- **Option A (Fast & Unsafe):** Putting all services together (Monolithic) allows direct communication, making it "blazing fast". However, it is unsafe because a single bug can crash the entire system.
- **Option B (Slow & Safe):** Separating services (Microkernel) isolates failures, making the system "super safe". However, the reliance on message passing makes it slower.
- **The Solution:** Modern systems like macOS use a **Hybrid Kernel** to balance this, keeping critical components monolithic for speed and others isolated for safety



## Challenge 2: Multiplicity (The Illusion of Parallelism)

- **The Problem:** The OS must figure out how to run hundreds of programs on a single CPU.
- **The Transformation:** It involves taking "dead code" (static programs on disk) and turning it into "living processes" in memory .
- **The Solution:** The OS creates an **illusion of parallelism**. By using **Context Switching**, it switches between processes so quickly that it appears they are all running simultaneously.

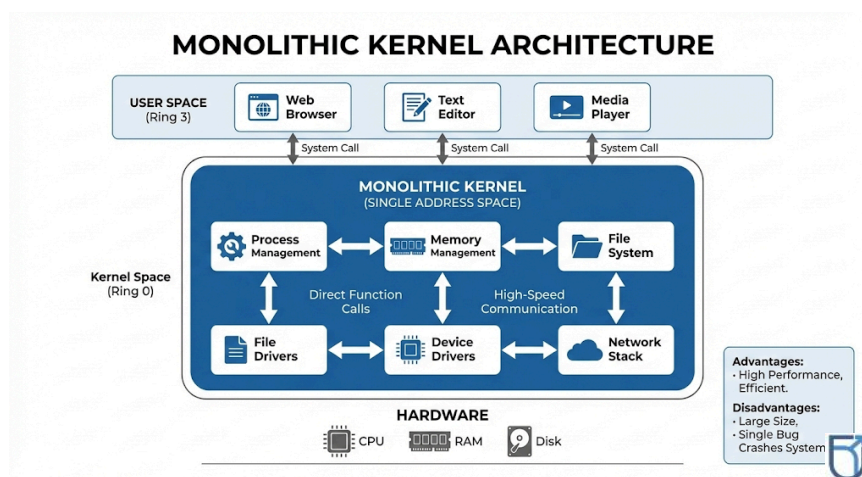
## Challenge 3: Management (Perfect Memory)

- **The Problem:** The OS must track and control every single thing happening in the system.
- **The Requirement:** To do this, the OS needs "perfect memory" of every resource and execution state.
- **The Solution:** It uses the **Process Control Block (PCB)**. The PCB acts as a data structure that contains *all* information about a specific process, allowing the OS to manage state transitions and scheduling effectively .

# OS Architecture Types

## Monolithic Kernel

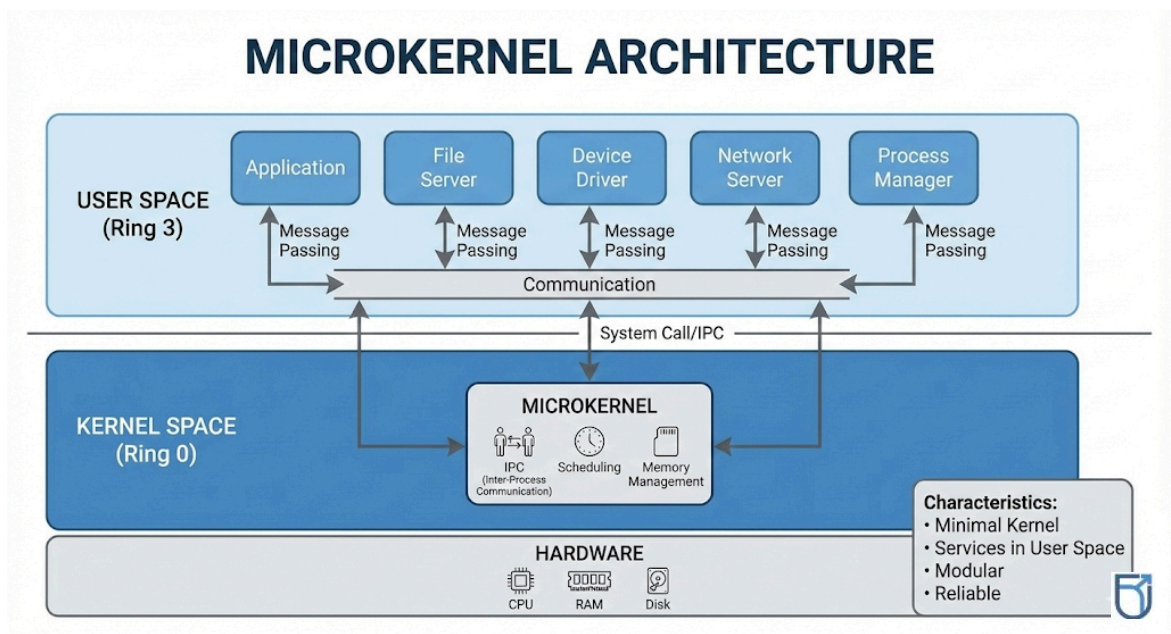
- **Definition:** Imagine a massive, single block of code where the entire OS lives in one place. All core services—process management, memory management, and critically, **all device drivers, file systems, and the networking stack**—run together in the privileged kernel space .



- **Philosophy:** "Fast & Unsafe." It operates like having your kitchen, bedroom, and bathroom all in one giant room without walls; moving between them is instant, but a fire in one corner burns down the whole house.
- **Advantages:**
  - **Blazing Speed:** Because every component is in the same address space, they communicate via **direct function calls** rather than complex message passing.
  - **Efficiency:** There is zero boundary overhead, making it highly efficient for raw processing.
- **Disadvantages:**
  - **Fragile Stability:** This is the biggest risk. A single bug—like a faulty graphics driver—can crash the **entire system**, leading to a "Kernel Panic" or the infamous "Blue Screen of Death" .
  - **Maintenance Nightmare:** The codebase is huge and complex, making it harder to debug and maintain over time.
- **Examples:** Linux (which helps manage complexity using **loadable kernel modules**), Traditional UNIX.

### Microkernel (The "Minimalist" Approach)

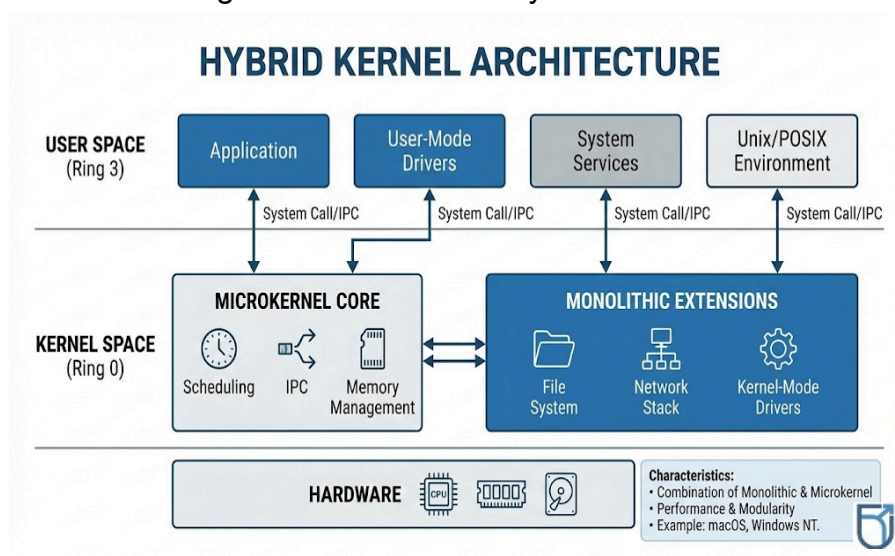
- **Definition:** A "tiny" kernel design that aggressively moves as much as possible out of the kernel. Most services run in **user space**, just like regular applications.



- **Kernel Responsibilities:** The kernel is stripped down to the bare essentials :
  - Memory management.
  - Process scheduling.
  - Inter-Process Communication (IPC).
- **User-Space Services:** Everything else—**file systems, device drivers, and networking stacks**—runs as separate, isolated user processes .
- **Advantages:**
  - **Reliability ("Super Safe"):** If a driver (like your audio driver) crashes, it's just a user process. You can simply restart that specific service without rebooting the whole machine. The system survives.
  - **Modularity:** It is much easier to extend, debug, and maintain because components are isolated.
- **Disadvantages:**
  - **Performance Overhead:** Since components are separated, every interaction requires **message passing** and **context switches** between user and kernel space, which is significantly slower than the direct calls used in monolithic kernels .
- **Examples:** QNX (often used in automotive systems like cars), MINIX, original Mach.

### Hybrid Kernel (The "Best of Both Worlds")

- **Definition:** This architecture represents an **"intelligent compromise."** It prioritizes **pragmatism over ideology** by combining the raw speed of monolithic designs with the modularity



- **Mechanism:**
  - **Critical services** stay in kernel space for maximum performance (speed where it matters).
  - **Non-critical services** move to user space for better stability (safety where possible).
- **Key Example: macOS XNU ("X is Not Unix"):**
  - **Mach Microkernel Core:** Handles the basics like memory management, IPC, and scheduling.
  - **BSD Monolithic Layer:** Sits on top to provide **POSIX APIs**, the networking stack, and file systems.
  - **IOKit:** Runs drivers in kernel space to ensure hardware interactions are fast.

### Exokernel (Brief)

- **Definition:** A radical architecture that minimizes abstractions entirely. It exposes the hardware directly to applications to squeeze out maximum performance .
- **Status:** It remains primarily **research-oriented** and is not used in mainstream consumer operating systems.

## Virtual Machines & Virtualization

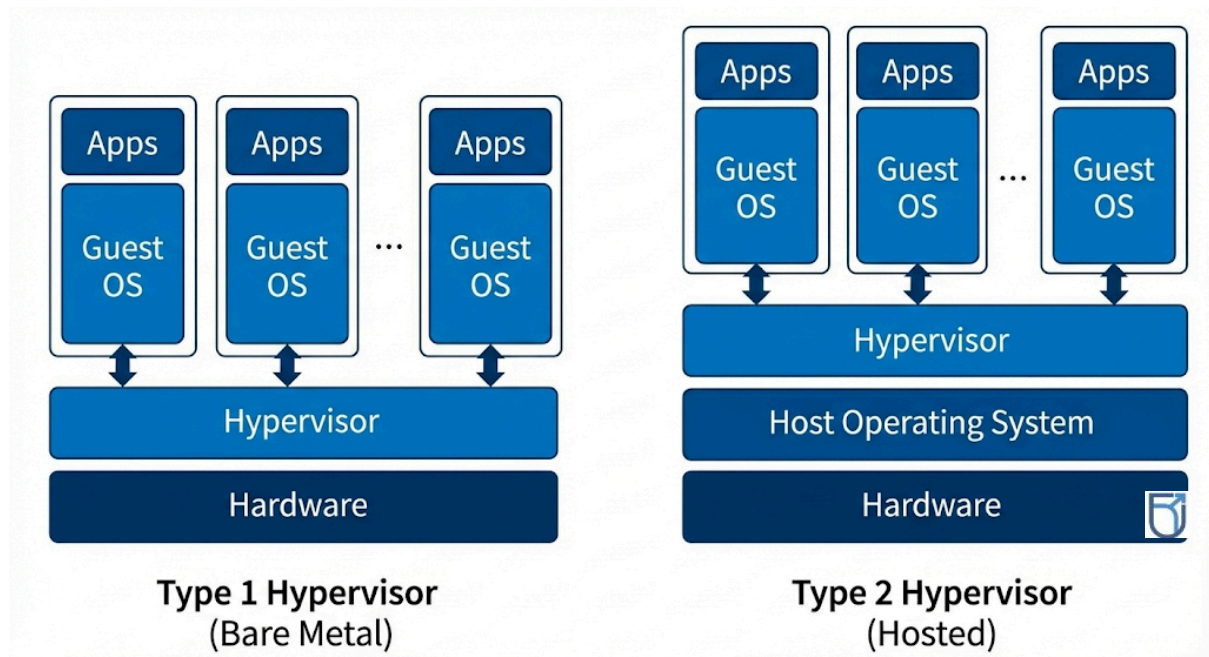
### Virtual Machines (The "Magic Trick")

- **Definition:** A Virtual Machine (VM) is essentially the **software emulation** of a physical computer.
- **The "New Problem":** What if you need to run Windows, Linux, and macOS all on a single physical machine? Or what if you want to run 50 servers on one piece of hardware?
- **Mechanism:** The VM creates an illusion where one physical computer acts as multiple virtual ones. Crucially, each VM **thinks it is alone** on real hardware, unaware that it is sharing resources .
- **Benefits:**
  - **Isolation:** It provides a perfect security sandbox. If a "guest" OS gets a virus or crashes, the host and other VMs remain safe.



- **Cloud Computing:** This is the foundation of the modern internet (AWS, Azure, Google Cloud). It allows one physical server to do the work of 50 virtual servers, maximizing efficiency.
- **Testing Environments:** Developers can test software in a safe, contained environment without risking their main computer.

### Hypervisors (The Managers)



The software that performs this magic—sitting between the hardware and the operating systems—is called a **Hypervisor** (or Virtual Machine Manager).

- **Type 1 – Bare-Metal (The "Pro" Grade)**
  - **How it runs:** These run **directly on the physical hardware**. There is no "host" operating system underneath slowing things down.
  - **Performance:** Because they interact directly with the hardware, they offer **maximum performance** and efficiency.
  - **Use Case:** Used extensively in data centers, enterprise servers, and cloud infrastructure where speed is critical.
  - **Examples:**
    - VMware ESXi.
    - Xen.
    - KVM (Kernel-based Virtual Machine).
    - Microsoft Hyper-V.

- **Apple Virtualization Framework** (Specifically designed for Apple Silicon performance).
- **Type 2 – Hosted (The "Developer" Grade)**
  - **How it runs:** These run as a standard application **on top of a host OS** (like running a program on macOS or Windows).
  - **Pros & Cons:** They are **easier to set up** (just install an app), but they suffer from **lower performance** because every operation has to go through the Host OS first.
  - **Use Case:** Perfect for personal use, such as a student running Windows on a Mac for a specific assignment, or for testing and development.
  - **Examples:**
    - VMware Fusion.
    - VirtualBox.
    - Parallels Desktop.

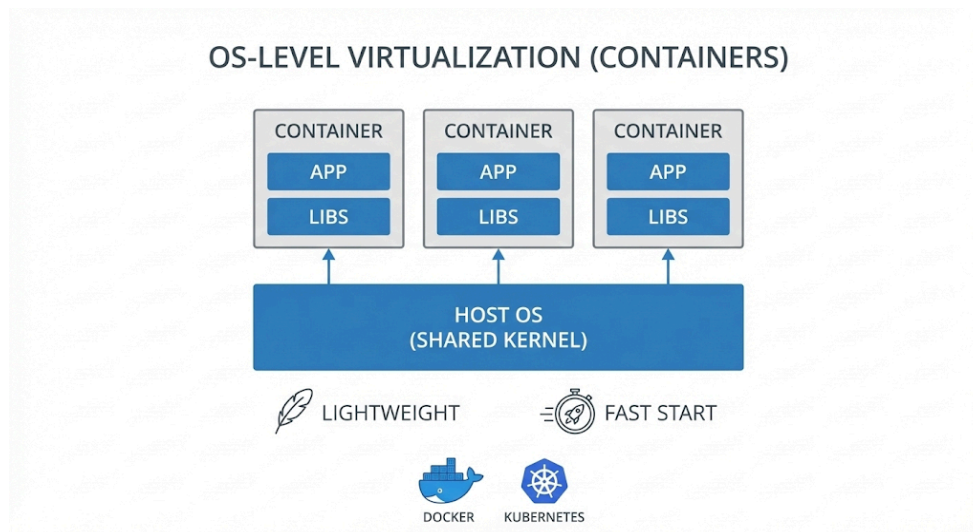
#### **Bonus: OS-Level Virtualization (Containers)**

- While VMs virtualize the hardware, **Containers** (like **Docker**) virtualize the OS itself. They allow applications to share the same kernel while keeping user spaces isolated, making them even more lightweight than VMs

## **OS-Level Virtualization (Containers)**

- **Concept:** Instead of simulating an entire physical machine with its own heavy OS (like a VM does), containers take a smarter approach. They **share the host operating system's kernel** while keeping the user space (apps and

libraries) completely isolated.



- **Why it's better:** Because they don't need to load a full OS for every app, they are incredibly **lightweight** and fast to start compared to traditional Virtual Machines.
- **Examples:**
  - **Docker:** The standard for building and running containers.
  - **Kubernetes:** A system for managing (orchestrating) massive numbers of containers.

## Program vs Process

| Feature           | Program                                                  | Process                                                 |
|-------------------|----------------------------------------------------------|---------------------------------------------------------|
| <b>Definition</b> | A static entity stored on the disk                       | A dynamic entity; a program currently in execution      |
| <b>Nature</b>     | Passive: It is just a file or dead code                  | Active: It is alive and performing tasks                |
| <b>Location</b>   | Stored on disk (secondary storage)                       | Loaded into RAM (main memory)                           |
| <b>Resources</b>  | Uses only disk space                                     | Actively uses CPU, memory, and I/O resources            |
| <b>State</b>      | Has no execution state (no program counter or registers) | Has execution states like Ready, Running, Waiting, etc. |
| <b>Lifespan</b>   | Permanent: Exists until the file is deleted              | Temporary: Exists only while it is executing            |



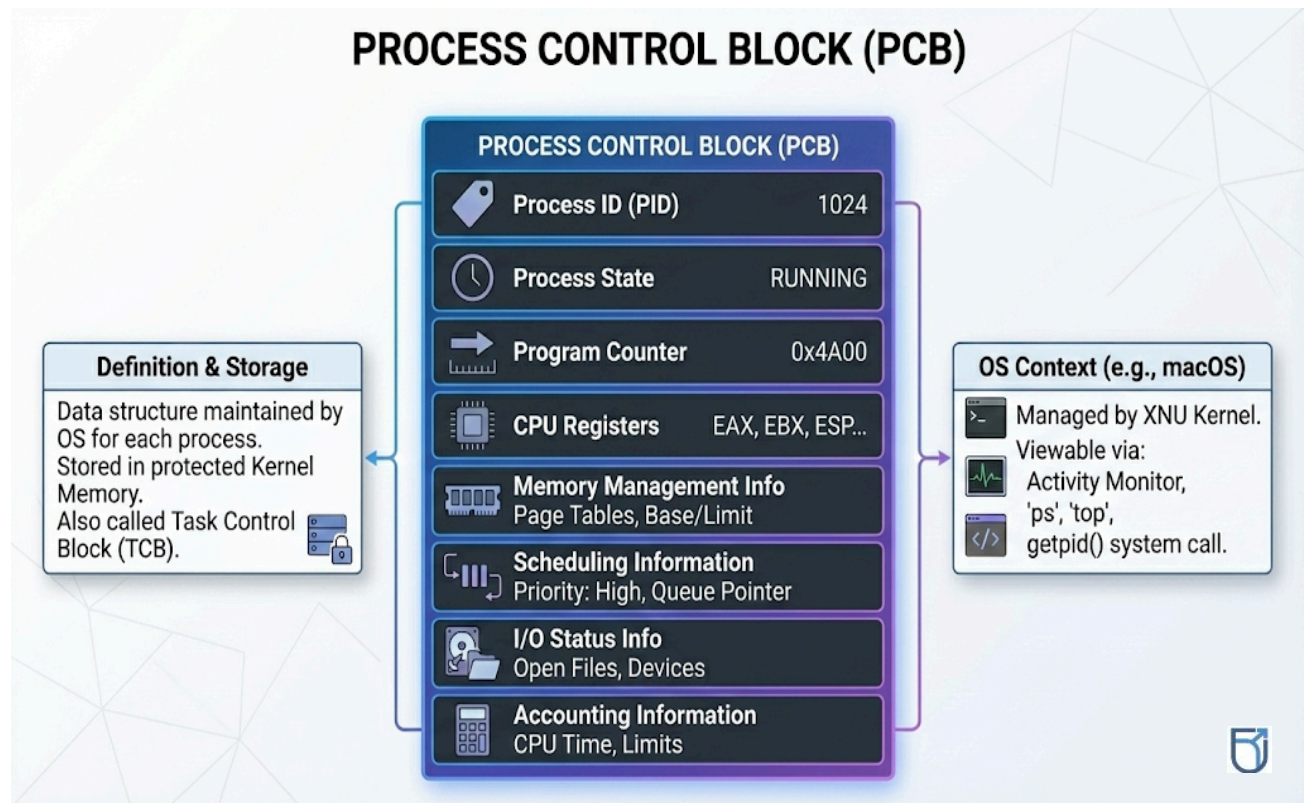
|                    |                                |                                           |
|--------------------|--------------------------------|-------------------------------------------|
| <b>Cardinality</b> | One single program file        | One program can create multiple processes |
| <b>Analogy</b>     | Like a recipe book on a shelf  | Like cooking the meal right now           |
| <b>Example</b>     | Safari app icon stored on disk | An open Safari window or tab              |

## Process Control Block (PCB)

### Definition

#### What is it?

The **Process Control Block (PCB)** is a fundamental **data structure maintained by the Operating System for every active process**.



## PCB Contents

| Field                       | Description                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Process ID (PID)</b>     | A unique number that identifies the process (similar to a Social Security Number).                                    |
| <b>Process State</b>        | The current status of the process (New, Ready, Running, Waiting, Terminated).                                         |
| <b>Program Counter (PC)</b> | A pointer to the memory address of the next instruction to be executed.                                               |
| <b>CPU Registers</b>        | Stores values of all CPU registers (accumulators, stack pointers, etc.) so data is not lost during context switching. |
| <b>Memory Info</b>          | Tracks memory management data such as page tables, base registers, and limit registers.                               |
| <b>Scheduling Info</b>      | Contains priority, pointers to scheduling queues, and allocated CPU time.                                             |
| <b>I/O Status</b>           | List of files currently open by the process and devices assigned to it.                                               |
| <b>Accounting Info</b>      | Statistics such as total CPU usage, time limits, and process numbers.                                                 |

---

## PCB in macOS

### **Management:**

In macOS, PCBs are managed by the **XNU kernel**.

### **Observation Tools:**

Information derived from the PCB can be viewed using:

- **GUI:** Activity Monitor
- **Terminal:** Commands like `ps` (process status) and `top`

# Process States & Lifecycle

## Five-State Model

### **New:**

The process is being created. The operating system is initializing the PCB.

### **Ready:**

The process is fully prepared to execute and is waiting in the **Ready Queue** for the CPU to become available.

### **Running:**

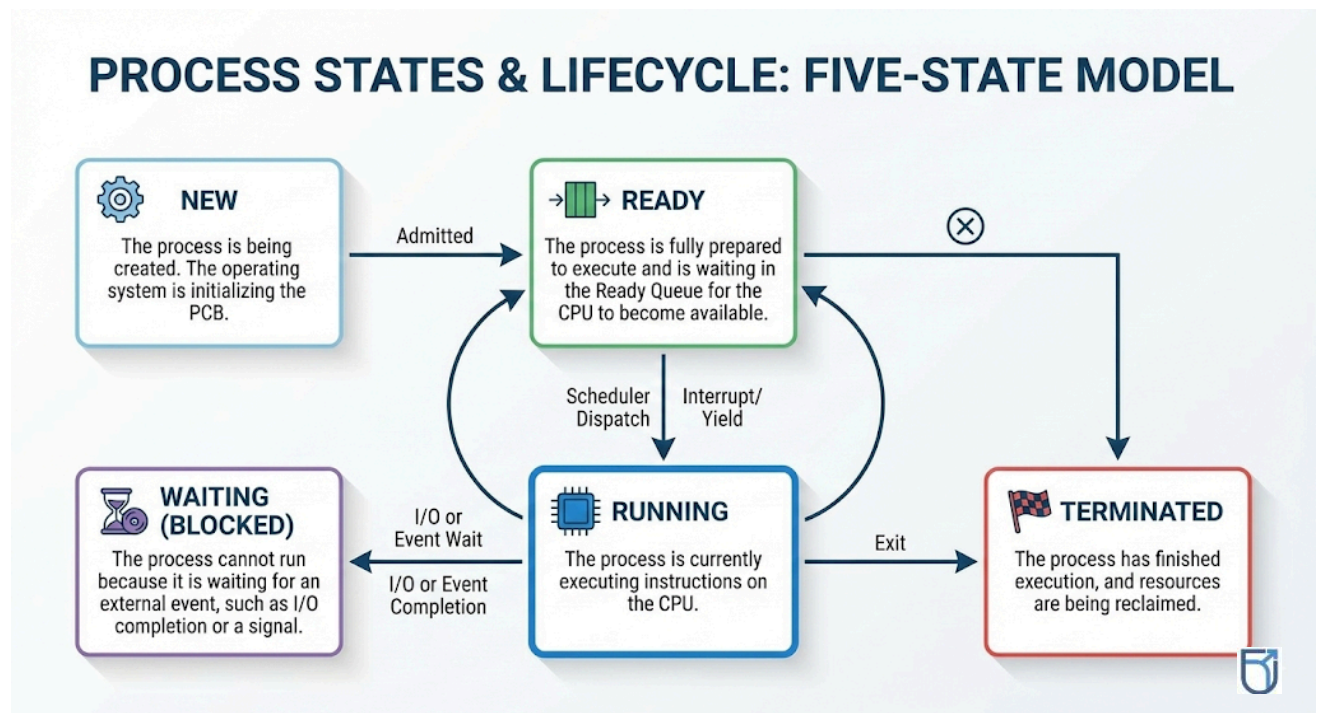
The process is currently executing instructions on the CPU.

### **Waiting (Blocked):**

The process cannot run because it is waiting for an external event, such as I/O completion or a signal.

### **Terminated:**

The process has finished execution, and the operating system is reclaiming its resources.



## State Transitions

### **New → Ready (Admission):**

The process is created and admitted to the ready queue.

### **Ready → Running (Dispatch):**

The scheduler selects the process and assigns it the CPU.

### Running → Ready (Preemption):

The process's time slice (quantum) expires, or a higher-priority process arrives. The OS pauses the process to allow another one to run.

### Running → Waiting (Block):

The process voluntarily pauses execution to wait for an event (for example, requesting a disk read).

### Waiting → Ready (Wakeup):

The event occurs (for example, disk I/O completes).

**Crucial Rule:** The process moves to the Ready state and never directly back to Running. It must wait in the ready queue again.

### Running → Terminated (Exit):

The process completes execution or crashes.

## Key Observations

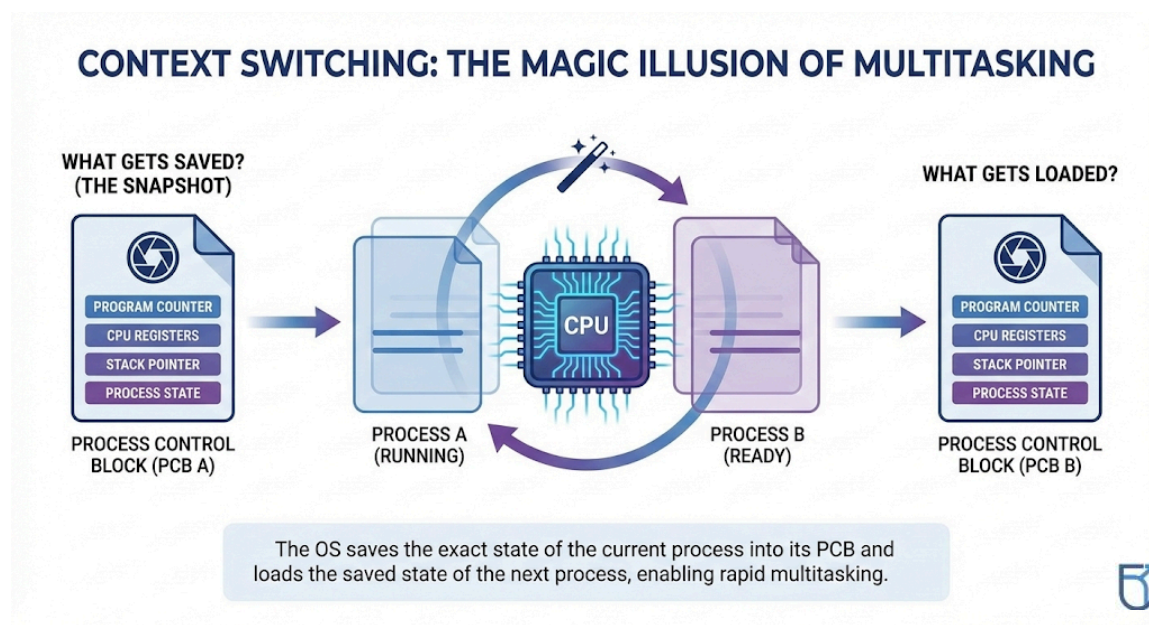
### Concurrency:

Multiple processes can be in the **Ready** state and multiple processes can be in the **Waiting** state at the same time.

### Singularity:

Only **one process per CPU core** can be in the **Running** state at any given moment.

## Context Switching (The "Magic Illusion")



## What is Context Switching?

### **Definition:**

Context switching is the essential mechanism where the OS saves the state of the current process and loads the saved state of the next process to be run.

### What Gets Saved? (The Snapshot)

To resume a process later without errors, the OS must save its exact hardware state into the Process Control Block (PCB):

- **Program Counter (PC):** Remembering exactly which instruction line to execute next.
- **CPU Registers:** All general-purpose registers and status flags.
- **Stack Pointer:** Tracking the process's memory stack.
- **Process State:** Updating the status (e.g., from Running to Ready).

### Overhead (The Cost of Doing Business)

Context switching is considered pure overhead because the CPU is doing administrative work rather than executing user code (useful work).

- **Time Cost:** A typical switch takes between 1–10 microseconds.
- **Hidden Costs:** It causes cache pollution and requires flushing the Translation Lookaside Buffer (TLB), which forces the CPU to reload memory data, slowing down performance.

### When It Occurs (The Triggers)

The OS performs a context switch during specific events:

- **Timer Interrupt:** The process's allocated time slice (quantum) has expired.
- **I/O Interrupt:** The process requests an I/O operation (like reading from a disk) and must wait for it to complete.
- **System Call:** The process asks the OS to perform a service.
- **Priority Preemption:** A higher-priority process (like a real-time task) becomes ready and takes over.



# Real-World Implications

## Performance Optimization

### Closing Apps:

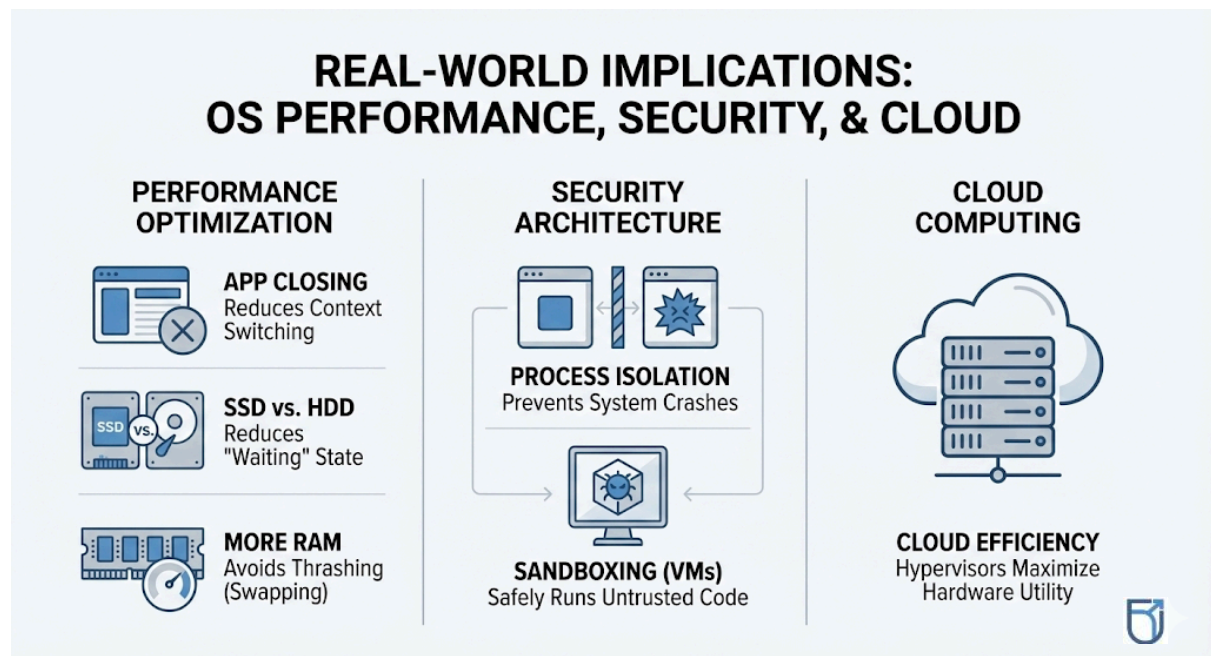
Closing unused apps helps because it reduces the number of processes competing for the CPU, thereby reducing context switching overhead.

### SSD vs. HDD:

SSDs make computers feel faster because they significantly reduce the time processes spend in the **Waiting** state (waiting for disk I/O).

### RAM:

More RAM allows more processes to stay resident in memory, avoiding **thrashing** (swapping to disk).



## Security Architecture

### Isolation:

Process isolation ensures that if one app crashes, it does not take down the whole system (a benefit of modern OS design).

### Sandboxing:

Virtual Machines are used to run untrusted code safely; if malware infects the VM, the host remains secure.

## Cloud Computing



### Efficiency:

Hypervisors are the secret sauce of the cloud (AWS, Azure), allowing providers to pack many virtual servers onto single physical hardware to maximize utility.

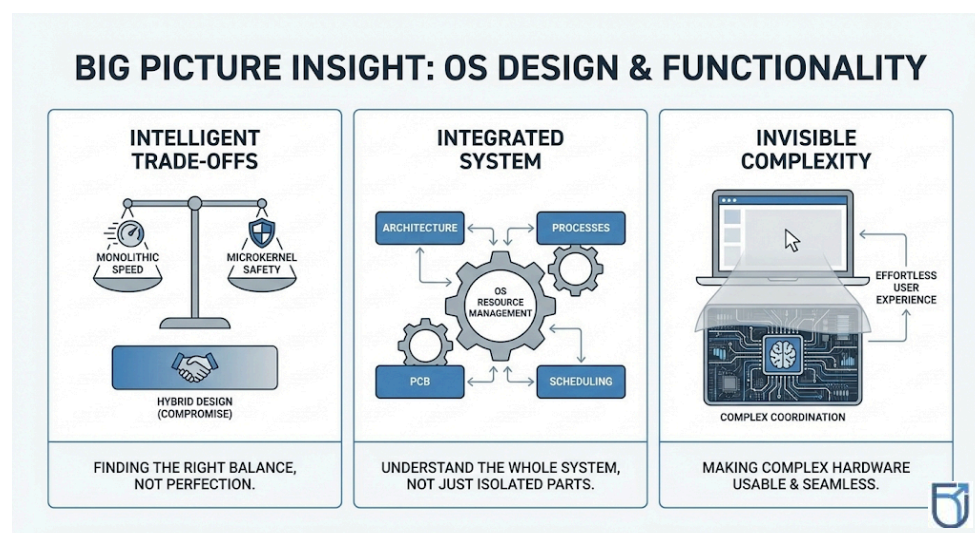
## Common Misconceptions (Busting Myths)

| Myth                                      | Reality                                                                                                                                      |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| "A program and a process are the same."   | False. A program is dead code on disk; a process is living execution with state and resources.                                               |
| "Context switching is free."              | False. It costs 1–10 microseconds of pure overhead and causes cache pollution.                                                               |
| "Processes jump from Waiting to Running." | False. A process must go to the Ready queue first to wait for the scheduler's permission.                                                    |
| "The OS just 'knows' what to do."         | False. The OS is just software; it explicitly tracks everything using data structures like the PCB. If the PCB is lost, the process is lost. |
| "VMs are fake and slow."                  | False. Type 1 hypervisors run directly on hardware and achieve near-native performance.                                                      |

## Big Picture Insight

### Intelligent Trade-offs:

Operating system design is not about finding one "perfect" solution; it is about finding the right compromise (for example, monolithic speed vs. microkernel safety resulting in a hybrid design).



**Integrated System:**

You cannot understand the OS by looking at parts in isolation. Architecture, processes, PCBs, and scheduling form one integrated system that solves the fundamental problem of resource management.

**Invisible Complexity:**

The ultimate goal of an OS is to make hardware usable. It performs complex coordination invisibly, making the impossible look effortless to the user.